



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

MEIM

MESTRADO EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
ENGENHARIA SOFTWARE

Implementação

Miguel Alcobia (50746)

Docente

Professor Luís Morgado

Janeiro, 2026

Índice

Índice	i
Lista de Figuras	ii
Lista de Códigos	iii
1 Introdução	1
1.1 Âmbito	1
1.2 Definições, Abreviaturas e Acrónimos	1
2 Implementação	3
2.1 Criação e População da Base de Dados	3
2.2 Apresentação	5
2.3 Acesso aos Dados	20
2.4 Domínio	29
3 Testes	46
Bibliografia	54

Lista de Figuras

1.1	Estrutura da implementação	2
-----	--------------------------------------	---

Lista de Códigos

1	Criação das tabelas da base de dados	3
2	População de algumas tabelas da base de dados	4
3	Página HTML exemplo	5
4	Implementação do Adaptador da Cliente	13
5	Classe Repositório Consumo SQL	20
6	Classe Repositório Hóspede SQL	21
7	Classe Repositório Quarto SQL	23
8	Classe Repositório Reserva SQL	26
9	Classe Serviço Reserva	29
10	Classe Serviço Hóspede	34
11	Entidade Hóspede	35
12	Entidade Reserva	37
13	Enumerado Estado da Reserva	38
14	Dados do Hóspede	38
15	Dados da Reserva	39
16	Adaptador Servidor (API)	40
17	Rotas da API (API)	44
18	Módulo de teste para criar reserva	46
19	Módulo de teste para registrar hóspede	49
20	Módulo de teste para listar serviços/consumos	50
21	Módulo de teste para listar quartos disponíveis	51

Capítulo 1

Introdução

Este documento apresenta a implemtenação realizada para o caso de utilização **Criar Reserva** e contém todo o código realizado, inclusive o dos respetivos testes.

1.1 Âmbito

Esta implemmentação insere-se no âmbito da realização do Projeto da unidade curricular Engenharia de Software, do Mestrado em Engenharia Informática e Multimédia do DEI do ISEL. [Morgado, 2025]

1.2 Definições, Abreviaturas e Acrónimos

Ver o documento da Especificação de Requisitos.

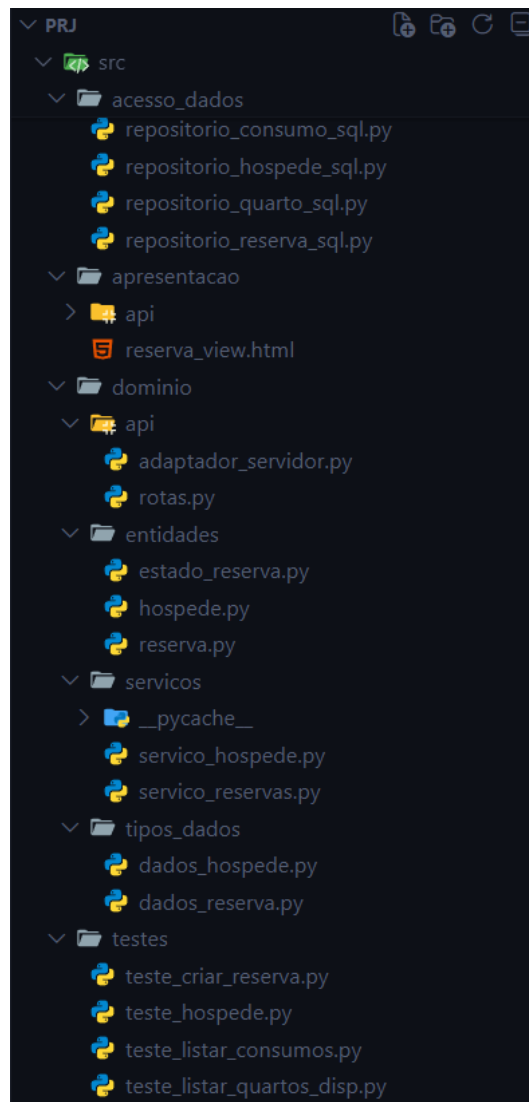


Figura 1.1: Estrutura da implementação

Capítulo 2

Implementação

2.1 Criação e População da Base de Dados

```
1 CREATE DATABASE IF NOT EXISTS hotel;
2 USE hotel;
3
4 -- Tabela hospede
5 CREATE TABLE hospede (
6     idHospede INT PRIMARY KEY AUTO_INCREMENT,
7     nomeCompleto VARCHAR(255) NOT NULL,
8     numDocumento VARCHAR(50) UNIQUE NOT NULL,
9     validadeDoc DATE,
10    birthDate DATE,
11    nacionalidade VARCHAR(100),
12    email VARCHAR(255),
13    telemovel VARCHAR(20),
14    preferencias TEXT
15 );
16
17 -- Tabela tipo_quarto
18 CREATE TABLE tipo_quarto (
19     tipo VARCHAR(20) PRIMARY KEY,
20     precoBase DECIMAL(10,2) NOT NULL
21 );
22
23 -- Tabela quarto
24 CREATE TABLE quarto (
25     numero INT PRIMARY KEY,
26     tipo VARCHAR(20) NOT NULL,
27     precoBase DECIMAL(10,2) NOT NULL,
28     estadoAtual ENUM('DISPONIVEL', 'OCUPADO', 'RESERVADO', '
EM_LIMPEZA', 'EM_MANUTENCAO') DEFAULT 'DISPONIVEL',
29     FOREIGN KEY (tipo) REFERENCES tipo_quarto(tipo)
```

```
30 );
31
32 -- Tabela servico
33 CREATE TABLE servico (
34     idServico INT PRIMARY KEY AUTO_INCREMENT,
35     nomeServico VARCHAR(255) NOT NULL,
36     preco DECIMAL(10,2) NOT NULL
37 );
38
39 -- Tabela reserva
40 CREATE TABLE reserva (
41     idReserva INT PRIMARY KEY AUTO_INCREMENT,
42     dataCriacao DATE NOT NULL,
43     dataEntradaPrev DATE NOT NULL,
44     dataSaidaPrev DATE NOT NULL,
45     estado ENUM('CONFIRMADA', 'CHECKIN_EFETUADO', '
CHECKOUT_EFETUADO', 'CANCELADA') NOT NULL,
46     valorTotalPrev DECIMAL(10,2),
47     notas TEXT,
48     idHospede INT NOT NULL,
49     numeroQuarto INT NOT NULL,
50     FOREIGN KEY (idHospede) REFERENCES hospede(idHospede),
51     FOREIGN KEY (numeroQuarto) REFERENCES quarto(numero)
52 );
53
54 -- Tabela de relacionamento entre reserva e servico (M:N)
55 -- Corrige-se um erro notado no Modelo de Dominio, pois esta
56 -- relacao estava definida como (1:N), o que nao faz sentido,
57 -- porque varias reservas deveriam poder usufruir do mesmo
servico
58 CREATE TABLE reserva_servico (
59     idReserva INT,
60     idServico INT,
61     PRIMARY KEY (idReserva, idServico),
62     FOREIGN KEY (idReserva) REFERENCES reserva(idReserva),
63     FOREIGN KEY (idServico) REFERENCES servico(idServico)
64 );
```

Código 1: Criação das tabelas da base de dados

```
1 -- Inserir tipos de quarto
2 INSERT INTO tipo_quarto (tipo, precoBase) VALUES
3 ('SOLTEIRO', 80.00),
4 ('CASAL', 120.00),
5 ('SUITE', 250.00);
6
7 -- Inserir quartos
8 -- Quartos Solteiro (101-110)
9 INSERT INTO quarto (numero, tipo, precoBase, estadoAtual)
```

```
VALUES
10 (101, 'SOLTEIRO', 80.00, 'DISPONIVEL'),
11 (102, 'SOLTEIRO', 80.00, 'DISPONIVEL'),
12 (103, 'SOLTEIRO', 80.00, 'DISPONIVEL');
13
14 -- Quartos Casal (201-210)
15 INSERT INTO quarto (numero, tipo, precoBase, estadoAtual)
    VALUES
16 (201, 'CASAL', 120.00, 'DISPONIVEL'),
17 (202, 'CASAL', 120.00, 'DISPONIVEL'),
18 (203, 'CASAL', 120.00, 'DISPONIVEL');
19
20 -- Quartos Suite (301-305)
21 INSERT INTO quarto (numero, tipo, precoBase, estadoAtual)
    VALUES
22 (301, 'SUITE', 250.00, 'DISPONIVEL'),
23 (302, 'SUITE', 250.00, 'DISPONIVEL'),
24 (303, 'SUITE', 250.00, 'DISPONIVEL');
25
26 -- Inserir serviços
27 INSERT INTO servico (nomeServico, preco) VALUES
28 ('Pequeno-Almoço', 15.00),
29 ('Lavanderia', 12.00),
30 ('Estacionamento', 10.00),
31 ('Massagem', 50.00),
32 ('Spa', 35.00);
```

Código 2: População de algumas tabelas da base de dados

2.2 Apresentação

```
1 <!DOCTYPE html>
2 <html lang="pt">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7     <title>Reserva de Hotel</title>
8     <style>
9         * {
10             box-sizing: border-box;
11             margin: 0;
12             padding: 0;
13             font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
14             sans-serif;
15         }
```

```
15     body {
16         background-color: #f5f7fa;
17         color: #333;
18         line-height: 1.6;
19         padding: 20px;
20     }
21
22     .container {
23         max-width: 1000px;
24         margin: 0 auto;
25         background-color: white;
26         border-radius: 12px;
27         box-shadow: 0 5px 20px rgba(0, 0, 0, 0.08);
28         overflow: hidden;
29     }
30
31     header {
32         background: linear-gradient(135deg, #1a3a5f 0%,
#2c5282 100%);
33         color: white;
34         padding: 25px 30px;
35         text-align: center;
36     }
37
38     h1 {
39         font-size: 28px;
40         margin-bottom: 8px;
41         font-weight: 600;
42     }
43
44     .subtitle {
45         font-size: 16px;
46         opacity: 0.9;
47         font-weight: 300;
48     }
49
50     .form-container {
51         padding: 30px;
52     }
53
54     .form-section {
55         margin-bottom: 35px;
56         padding-bottom: 25px;
57         border-bottom: 1px solid #eaeaea;
58     }
59
60     .section-title {
61         font-size: 20px;
62         color: #2c5282;
```

```
63         margin-bottom: 20px;
64         padding-bottom: 8px;
65         border-bottom: 2px solid #e2e8f0;
66         font-weight: 600;
67     }
68
69     .form-row {
70         display: flex;
71         flex-wrap: wrap;
72         margin: 0 -10px;
73     }
74
75     .form-group {
76         flex: 1 0 300px;
77         margin: 0 10px 20px;
78         min-width: 0;
79     }
80
81     label {
82         display: block;
83         margin-bottom: 8px;
84         font-weight: 500;
85         color: #4a5568;
86     }
87
88     input, select, textarea {
89         width: 100%;
90         padding: 12px 15px;
91         border: 1px solid #cbd5e0;
92         border-radius: 6px;
93         font-size: 16px;
94         transition: border 0.3s, box-shadow 0.3s;
95     }
96
97     input:focus, select:focus, textarea:focus {
98         outline: none;
99         border-color: #4299e1;
100        box-shadow: 0 0 0 3px rgba(66, 153, 225, 0.15);
101    }
102
103    textarea {
104        min-height: 120px;
105        resize: vertical;
106        font-family: inherit;
107    }
108
109    .dates-container {
110        display: flex;
111        gap: 20px;
```

```
112     }
113
114     .date-group {
115         flex: 1;
116     }
117
118     .button-container {
119         text-align: center;
120         margin-top: 30px;
121         padding-top: 20px;
122         border-top: 1px solid #eaeaea;
123     }
124
125     .confirm-button {
126         background: linear-gradient(to right, #2c5282, #1
a3a5f);
127         color: white;
128         border: none;
129         padding: 16px 40px;
130         font-size: 18px;
131         border-radius: 8px;
132         cursor: pointer;
133         font-weight: 600;
134         transition: all 0.3s;
135         box-shadow: 0 4px 12px rgba(28, 63, 102, 0.2);
136     }
137
138     .confirm-button:hover {
139         background: linear-gradient(to right, #1a3a5f, #2
c5282);
140         transform: translateY(-2px);
141         box-shadow: 0 6px 16px rgba(28, 63, 102, 0.3);
142     }
143
144     .required::after {
145         content: " *";
146         color: #e53e3e;
147     }
148
149     .form-note {
150         font-size: 14px;
151         color: #718096;
152         margin-top: 5px;
153         font-style: italic;
154     }
155
156     footer {
157         text-align: center;
158         padding: 20px;
```

```
159         color: #718096;
160         font-size: 14px;
161         background-color: #f8fafc;
162         border-top: 1px solid #e2e8f0;
163     }
164
165     @media (max-width: 768px) {
166         .dates-container {
167             flex-direction: column;
168             gap: 0;
169         }
170
171         .form-group {
172             flex: 1 0 100%;
173         }
174
175         .form-container {
176             padding: 20px;
177         }
178     }
179 </style>
180 </head>
181 <body>
182     <div class="container">
183         <header>
184             <h1>Reserva de Hotel</h1>
185         </header>
186
187         <div class="form-container">
188             <form action="#" method="post">
189                 <!-- Informações Pessoais -->
190                 <div class="form-section">
191                     <h2 class="section-title">Informações
192                     Pessoais</h2>
193
194                     <div class="form-row">
195                         <div class="form-group">
196                             <label for="nome_completo" class="
197                             required">Nome Completo</label>
198                             <input type="text" id="
199                             nome_completo" name="nome_completo" required placeholder="
200                             Ex: João Silva Santos">
201                         </div>
202
203                         <div class="form-group">
204                             <label for="cc_number" class="
205                             required">Número do Cartão de Cidadão</label>
206                             <input type="text" id="cc_number"
207                             name="cc_number" required placeholder="Ex: 12345678">
208                         </div>
209                     </div>
210                 </div>
211             </form>
212         </div>
213     </div>
214 </body>
215 </html>
```

```

202         </div>
203
204         <div class="form-row">
205             <div class="form-group">
206                 <label for="doc_validade" class="
207                 required">Data de Validade do Documento</label>
208                 <input type="date" id="
209                 doc_validade" name="doc_validade" required>
210             </div>
211
212             <div class="form-group">
213                 <label for="data_nascimento"
214                 class="required">Data de Nascimento</label>
215                 <input type="date" id="
216                 data_nascimento" name="data_nascimento" required>
217             </div>
218         </div>
219
220         <div class="form-row">
221             <div class="form-group">
222                 <label for="nacionalidade" class="
223                 "required">Nacionalidade</label>
224                 <select id="nacionalidade" name="
225                 nacionalidade" required>
226                     <option value="">Selecione
227                     uma nacionalidade</option>
228                     <option value="pt">Portuguesa
229                     </option>
230                     <option value="br">Brasileira
231                     </option>
232                     <option value="es">Espanhola<
233                     /option>
234                     <option value="fr">Francesa</
235                     option>
236                     <option value="it">Italiana</
237                     option>
238                     <option value="de">Alemã</
239                     option>
240                     <option value="uk">Inglesa</
241                     option>
242                     <option value="us">Americana<
243                     /option>
244                     <option value="outra">Outra</
245                     option>
246                 </select>
247             </div>
248         </div>
249     </div>

```



```

235         <!-- Informações de Contacto -->
236         <div class="form-section">
237             <h2 class="section-title">Informações de
Contacto</h2>
238             <div class="form-row">
239                 <div class="form-group">
240                     <label for="email" class="
required">E-mail</label>
241                     <input type="email" id="email"
name="email" required placeholder="exemplo@email.com">
242                 </div>
243
244                 <div class="form-group">
245                     <label for="telemovel" class="
required">Telemóvel</label>
246                     <input type="tel" id="telemovel"
name="telemovel" required placeholder="Ex: 912345678">
247                 </div>
248             </div>
249         </div>
250
251         <!-- Botão de Registo -->
252         <div class="button-container">
253             <button type="submit" class="confirm-
button">Registar Hóspede</button>
254         </div>
255
256         <!-- Detalhes da Reserva -->
257         <div class="form-section">
258             <h2 class="section-title">Detalhes da
Reserva</h2>
259             <div class="form-row">
260                 <div class="form-group">
261                     <label for="select_quartos" class
="required">Quarto</label>
262                     <select id="select_quartos" name=
"select_quartos" required>
263                         <option value="">Selecione um
tipo de quarto</option>
264                         <option value="single">Quarto
Solteiro - 80 /noite</option>
265                         <option value="double">Quarto
Casal - 120 /noite</option>
266                         <option value="suite">Suíte -
250 /noite</option>
267                     </select>
268                 </div>
269
270                 <div class="form-group">

```

```

271         <label for="select_servicos">
Serviços Adicionais</label>
272         <select id="select_servicos" name
="select_servicos">
273             <option value="">Selecione
serviços adicionais</option>
274             <option value="breakfast">
Pequeno-almoço ( 15 /dia)</option>
275             <option value="lavandaria">
Lavandaria ( 12 /dia)</option>
276             <option value="parking">
Estacionamento ( 10 /dia)</option>
277             <option value="massagem">
Massagem ( 50 /dia)</option>
278             <option value="spa">Acesso ao
SPA ( 35 /dia)</option>
279         </select>
280         <p class="form-note">Pode deixar
em branco se o cliente não desejar serviços adicionais</p>
281     </div>
282 </div>
283
284     <div class="form-row">
285         <div class="form-group">
286             <div class="dates-container">
287                 <div class="date-group">
288                     <label for="data_entrada"
class="required">Data de Entrada</label>
289                     <input type="date" id="
data_entrada" name="data_entrada" required>
290                 </div>
291
292                 <div class="date-group">
293                     <label for="data_saida"
class="required">Data de Saída</label>
294                     <input type="date" id="
data_saida" name="data_saida" required>
295                 </div>
296             </div>
297         </div>
298     </div>
299 </div>
300
301     <!-- Observações -->
302     <div class="form-section">
303         <h2 class="section-title">Observações</h2
>
304         <div class="form-row">
305             <div class="form-group">

```

```

306         <label for="observacoes">Observaç
ões Adicionais</label>
307         <textarea id="observacoes" name="
observacoes" placeholder="Por favor, indique quaisquer
necessidades especiais, pedidos ou informações adicionais
para a estadia do cliente..."></textarea>
308         <p class="form-note">Ex: Cuidado
na alimentação, acesso para pessoas com mobilidade
reduzida, etc.</p>
309     </div>
310 </div>
311 </div>
312
313     <!-- Botão de Confirmação -->
314     <div class="button-container">
315         <button type="submit" class="confirm-
button">Confirmar Reserva</button>
316         <p class="form-note" style="margin-top:
15px;">Campos marcados com * são obrigatórios</p>
317     </div>
318 </form>
319 </div>
320 </div>
321 <script src="api/adaptador_cliente.js"></script>
322 </body>
323 </html>

```

Código 3: Página HTML exemplo

```

1 // Adaptador do cliente para fazer chamadas à API
2 class AdaptadorCliente {
3     constructor() {
4         // URL da API
5         this.baseURL = 'http://localhost:5008/api';
6     }
7
8     /**
9      * Regista um novo hóspede (Pedido POST)
10     * @param {Object} hospedeData - Dados do hóspede
11     * @returns {Promise<Object>} Resposta da API
12     */
13     async registrarHospede(hospedeData) {
14         try {
15             const response = await fetch(`${this.baseURL}/
registar_hosp`, {
16                 method: 'POST',
17                 headers: {
18                     'Content-Type': 'application/json',
19                 },

```

```
20         body: JSON.stringify(hospedeData)
21     });
22
23     if (!response.ok) {
24         throw new Error('Erro HTTP: ${response.status
25     }');
26     }
27
28     return await response.json();
29 } catch (error) {
30     console.error('Erro ao registrar hóspede:', error)
31 ;
32     throw error;
33 }
34
35 /**
36  * Consulta hóspede por email (Pedido GET)
37  * @param {string} email - Email do hóspede
38  * @returns {Promise<Object>} Dados do hóspede
39  */
40 async consultarHospede(email) {
41     try {
42         const response = await fetch(`${this.baseURL}/
43         consulta_hospede?email=${encodeURIComponent(email)}`);
44
45         if (!response.ok) {
46             throw new Error('Erro HTTP: ${response.status
47         }');
48         }
49
50         return await response.json();
51     } catch (error) {
52         console.error('Erro ao consultar hóspede:', error
53     );
54     throw error;
55 }
56
57 /**
58  * Confirma uma reserva (Pedido POST)
59  * @param {Object} reservaData - Dados da reserva
60  * @returns {Promise<Object>} Resposta da API
61  */
62 async confirmarReserva(reservaData) {
63     try {
64         const response = await fetch(`${this.baseURL}/
65         confirm_reserva`, {
66             method: 'POST',
```

```
63         headers: {
64             'Content-Type': 'application/json',
65         },
66         body: JSON.stringify(reservaData)
67     });
68
69     if (!response.ok) {
70         throw new Error('Erro HTTP: ${response.status
71     }');
72     }
73
74     return await response.json();
75 } catch (error) {
76     console.error('Erro ao confirmar reserva:', error
77 );
78     throw error;
79 }
80
81 /**
82  * Obtém quartos disponíveis (pedido GET)
83  * @returns {Promise<Array>} Lista de quartos disponíveis
84  */
85 async obterQuartosDisponiveis() {
86     try {
87         const response = await fetch(`${this.baseUrl}/
88         quartos_disponiveis`);
89
90         if (!response.ok) {
91             throw new Error('Erro HTTP: ${response.status
92         }');
93         }
94
95         return await response.json();
96     } catch (error) {
97         console.error('Erro ao obter quartos disponíveis:
98         ', error);
99         throw error;
100     }
101 }
102
103 // instância para usar na página HTML
104 const adaptadorCliente = new AdaptadorCliente();
105
106 // Função para lidar com o registo do hóspede
107 async function handleRegistrarHospede(event) {
108     // Prevene envio padrão do formulário
109     event.preventDefault();
```

```
107
108     try {
109         // Junta os dados do form
110         const hospedeData = {
111             nomeCompleto: document.getElementById('
112 nome_completo').value,
113             numDocumento: document.getElementById('cc_number'
114 ).value,
115             validadeDoc: document.getElementById('
116 doc_validade').value,
117             birthDate: document.getElementById('
118 data_nascimento').value,
119             nacionalidade: document.getElementById('
120 nacionalidade').value,
121             email: document.getElementById('email').value,
122             telemovel: document.getElementById('telemovel').
123 value,
124             preferencias: ''
125         };
126
127         // Validar campos obrigatórios
128         const camposObrigatorios = [
129             'nomeCompleto', 'numDocumento', 'validadeDoc',
130             'birthDate', 'nacionalidade', 'email', 'telemovel'
131         ];
132
133         for (const campo of camposObrigatorios) {
134             if (!hospedeData[campo]) {
135                 alert('Por favor, preencha o campo ${campo}');
136             }
137             return;
138         }
139
140         // Chamar a API para registrar
141         const resultado = await adaptadorCliente.
142 registrarHospede(hospedeData);
143
144         if (resultado.success) {
145             alert('Hóspede registado com sucesso! ID: ' +
146 resultado.idHospede);
147         } else {
148             alert('Erro: ' + resultado.message);
149         }
150     } catch (error) {
151         alert('Erro ao registrar hóspede: ' + error.message);
152     }
153 }
```

```
146
147 // Função para lidar com a confirmação da reserva
148 async function handleConfirmarReserva(event) {
149     event.preventDefault();
150
151     try {
152         // dados da reserva
153         const idHospede = sessionStorage.getItem('idHospede')
154     ;
155         if (!idHospede) {
156             alert('Por favor, registre o hóspede primeiro');
157             return;
158         }
159
160         // dados do quarto selecionado
161         const quartoSelect = document.getElementById('
162 select_quartos');
163         const quartoValue = quartoSelect.value;
164
165         // Extrair número do quarto
166         let numeroQuarto = null;
167         let tipoQuarto = null;
168
169         if (quartoValue.includes('-')) {
170             [numeroQuarto, tipoQuarto] = quartoValue.split('-
171 ');
172         } else {
173             tipoQuarto = quartoValue;
174         }
175
176         const reservaData = {
177             idHospede: parseInt(idHospede),
178             numeroQuarto: numeroQuarto ? parseInt(
179 numeroQuarto) : null,
180             tipoQuarto: tipoQuarto,
181             servicosAdicionais: document.getElementById('
182 select_servicos').value,
183             dataEntradaPrev: document.getElementById('
184 data_entrada').value,
185             dataSaidaPrev: document.getElementById('
186 data_saida').value,
187             notas: document.getElementById('observacoes').
188 value
189         };
190
191         // Validar campos obrigatórios
192         if (!reservaData.tipoQuarto || !reservaData.
193 dataEntradaPrev || !reservaData.dataSaidaPrev) {
194             alert('Por favor, preencha todos os campos
```

```
obrigatórios da reserva');
186         return;
187     }
188
189     // Calcular número de dias
190     const dataEntrada = new Date(reservaData.
dataEntradaPrev);
191     const dataSaida = new Date(reservaData.dataSaidaPrev)
;
192     const diffTime = Math.abs(dataSaida - dataEntrada);
193     const diffDays = Math.ceil(diffTime / (1000 * 60 * 60
* 24));
194
195     if (diffDays <= 0) {
196         alert('A data de saída deve ser posterior à data
de entrada');
197         return;
198     }
199
200     reservaData.numeroDias = diffDays;
201
202     // Chamar a API para confirmar reserva
203     const resultado = await adaptadorCliente.
confirmarReserva(reservaData);
204
205     if (resultado.success) {
206         alert('Reserva confirmada com sucesso!\nID da
Reserva: ${resultado.idReserva}\nValor Total: $ {
resultado.valorTotalPrev.toFixed(2)}');
207
208         // Limpar sessão e formulário
209         sessionStorage.removeItem('idHospede');
210         document.querySelector('form').reset();
211     } else {
212         alert('Erro: ' + resultado.message);
213     }
214 } catch (error) {
215     alert('Erro ao confirmar reserva: ' + error.message);
216 }
217 }
218
219 // Carregar quartos disponíveis quando a página carrega
220 async function carregarQuartosDisponiveis() {
221     try {
222         const quartos = await adaptadorCliente.
obterQuartosDisponiveis();
223         const selectQuartos = document.getElementById('
select_quartos');
224
```



```
225     // Limpar opções existentes (menos a primeira)
226     while (selectQuartos.options.length > 1) {
227         selectQuartos.remove(1);
228     }
229
230     // Adicionar quartos disponíveis
231     quartos.forEach(quarto => {
232         const option = document.createElement('option');
233         option.value = `${quarto.numero}-${quarto.tipo}`;
234         option.textContent = `Quarto ${quarto.numero} - $
{quarto.tipo.toUpperCase()} - $ {quarto.precoBase}/noite
`;
235         selectQuartos.appendChild(option);
236     });
237 } catch (error) {
238     console.error('Erro ao carregar quartos:', error);
239 }
240 }
241
242 // Adicionar evento ao botão de registrar hóspede
243 document.addEventListener('DOMContentLoaded', function() {
244     const form = document.querySelector('form');
245
246     // Encontrar todos os botões de confirm
247     const botoesConfirmar = document.querySelectorAll('.
confirm-button');
248
249     if (botoesConfirmar.length > 0 && form) {
250         // Primeiro botão - Registrar Hóspede
251         botoesConfirmar[0].addEventListener('click', function
(event) {
252             if (this.textContent.includes('Registrar Hóspede'))
) {
253                 handleRegistrarHospede(event);
254             }
255         });
256
257         // Segundo botão - Confirmar Reserva
258         if (botoesConfirmar.length > 1) {
259             botoesConfirmar[1].addEventListener('click',
function(event) {
260                 if (this.textContent.includes('Confirmar
Reserva')) {
261                     handleConfirmarReserva(event);
262                 }
263             });
264         }
265     }
266 }
```

```
267 // Carregar quartos disponíveis
268 carregarQuartosDisponiveis();
269 });
```

Código 4: Implementação do Adaptador da Cliente

2.3 Acesso aos Dados

```
1 import mysql.connector
2
3 class RepositorioConsumoSQL:
4
5     def __init__(self):
6         # Configuração da conexão com o MySQL
7         self.connection = mysql.connector.connect(
8             host='localhost',
9             user='root',
10            password='root',
11            database='hotel'
12        )
13        self.cursor = self.connection.cursor(dictionary=True)
14
15    def listar_consumos(self):
16        try:
17            # Query para listar quartos disponíveis
18            query = """
19            SELECT *
20            FROM servico
21            """
22
23            self.cursor.execute(query)
24            servicos = self.cursor.fetchall()
25            return servicos
26
27        except mysql.connector.Error as err:
28            print(f"Erro ao listar quartos disponíveis: {err}")
29
30        return []
31
32    def criar(self, servico):
33        raise NotImplementedError
34
35    def eliminar(self, servico):
36        raise NotImplementedError
```

Código 5: Classe Repositório Consumo SQL

```
1 import mysql.connector
2 from datetime import datetime
3
4 class RepositorioHospedeSQL:
5     def __init__(self):
6         # Configuração da conexão com o MySQL
7         self.connection = mysql.connector.connect(
8             host='localhost',
9             user='root',
10            password='root',
11            database='hotel'
12        )
13        self.cursor = self.connection.cursor(dictionary=True)
14
15    def obter_hospede(self, num_doc):
16        """Obtém hóspede por número de documento"""
17        try:
18            query = "SELECT * FROM hospede WHERE numDocumento
19            = %s"
20            self.cursor.execute(query, (num_doc,))
21            return self.cursor.fetchone()
22        except Exception as e:
23            print(f"Erro ao obter hóspede: {e}")
24            raise
25
26    def obter_hospede_por_email(self, email):
27        """Obtém hóspede por email (método novo)"""
28        try:
29            query = "SELECT * FROM hospede WHERE email = %s"
30            self.cursor.execute(query, (email,))
31            return self.cursor.fetchone()
32        except Exception as e:
33            print(f"Erro ao obter hóspede por email: {e}")
34            raise
35
36    def verificar_documento(self, num_doc):
37        """Verifica se documento já existe"""
38        try:
39            query = "SELECT COUNT(*) as count FROM hospede
40            WHERE numDocumento = %s"
41            self.cursor.execute(query, (num_doc,))
42            result = self.cursor.fetchone()
43            return result['count'] > 0
44        except Exception as e:
45            print(f"Erro ao verificar documento: {e}")
46            raise
47
48    def registrar(self, hospede):
```

```
47     """Regista um novo hóspede"""
48     try:
49         query = """
50             INSERT INTO hospede
51             (nomeCompleto, numDocumento, validadeDoc,
52              birthDate,
53              nacionalidade, email, telemovel, preferencias)
54             VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
55             """
56
57         valores = (
58             hospede['nomeCompleto'],
59             hospede['numDocumento'],
60             hospede['validadeDoc'],
61             hospede['birthDate'],
62             hospede['nacionalidade'],
63             hospede['email'],
64             hospede['telemovel'],
65             hospede.get('preferencias', '')
66         )
67
68         self.cursor.execute(query, valores)
69         self.connection.commit()
70
71         # Obter o ID do hóspede inserido
72         id_hospede = self.cursor.lastrowid
73
74         return {
75             'success': True,
76             'message': 'Hóspede registado com sucesso',
77             'idHospede': id_hospede
78         }
79
80     except mysql.connector.Error as err:
81         # Se for erro de duplicação
82         if err.errno == 1062: # ER_DUP_ENTRY
83             return {
84                 'success': False,
85                 'message': 'Documento ou email já está
86                 registado'
87             }
88         else:
89             print(f"Erro MySQL ao registar: {err}")
90             return {
91                 'success': False,
92                 'message': f'Erro de banco de dados: {err}'
93             }
94     except Exception as e:
```

```
93         print(f"Erro ao registrar hóspede: {e}")
94         return {
95             'success': False,
96             'message': f'Erro interno: {str(e)}'
97         }
98
99     def eliminar(self, num_doc):
100         """Elimina um hóspede pelo número de documento"""
101         try:
102             query = "DELETE FROM hospede WHERE numDocumento =
103             %s"
104             self.cursor.execute(query, (num_doc,))
105             self.connection.commit()
106
107             return self.cursor.rowcount > 0
108
109         except Exception as e:
110             print(f"Erro ao eliminar hóspede: {e}")
111             raise
```

Código 6: Classe Repositório Hóspede SQL

```
1 import mysql.connector
2
3 class RepositorioQuartoSQL:
4
5     def __init__(self):
6         # Configuração da conexão com o MySQL
7         self.connection = mysql.connector.connect(
8             host='localhost',
9             user='root',
10            password='root',
11            database='hotel'
12        )
13        self.cursor = self.connection.cursor(dictionary=True)
14
15    def listar_disponiveis(self):
16        try:
17            # Query para listar quartos disponíveis
18            query = """
19            SELECT q.numero, q.tipo, q.precoBase, q.
20            estadoAtual
21            FROM quarto q
22            WHERE q.estadoAtual = 'DISPONIVEL'
23            ORDER BY q.numero
24            """
25
26            self.cursor.execute(query)
27            quartos = self.cursor.fetchall()
```

```
27         return quartos
28
29     except mysql.connector.Error as err:
30         print(f"Erro ao listar quartos disponíveis: {err}")
31
32     return []
33
34     def listar_por_tipo(self, tipo_quarto):
35         """Lista quartos por tipo"""
36         try:
37             query = """
38             SELECT q.numero, q.tipo, q.precoBase, q.
39             estadoAtual
40             FROM quarto q
41             WHERE q.tipo = %s AND q.estadoAtual = 'DISPONIVEL
42             ,
43             ORDER BY q.numero
44             """
45
46             self.cursor.execute(query, (tipo_quarto,))
47             quartos = self.cursor.fetchall()
48             return quartos
49
50         except mysql.connector.Error as err:
51             print(f"Erro ao listar quartos por tipo: {err}")
52             return []
53
54     def obter_quarto(self, numero_quarto):
55         """Obtém informações de um quarto específico"""
56         try:
57             query = """
58             SELECT q.numero, q.tipo, q.precoBase, q.
59             estadoAtual
60             FROM quarto q
61             WHERE q.numero = %s
62             """
63
64             self.cursor.execute(query, (numero_quarto,))
65             quarto = self.cursor.fetchone()
66             return quarto
67
68         except mysql.connector.Error as err:
69             print(f"Erro ao obter quarto: {err}")
70             return None
71
72     def atualizar_estado(self, numero_quarto, novo_estado):
73         """Atualiza o estado de um quarto"""
74         try:
75             query = """
```

```
72         UPDATE quarto
73         SET estadoAtual = %s
74         WHERE numero = %s
75         """
76
77         self.cursor.execute(query, (novo_estado,
78 numero_quarto))
79         self.connection.commit()
80         return True
81
82     except mysql.connector.Error as err:
83         print(f"Erro ao atualizar estado do quarto: {err}")
84
85         return False
86
87     def listar_ativos(self):
88         """Lista todos os quartos ativos"""
89         try:
90             query = """
91             SELECT q.numero, q.tipo, q.precoBase, q.
92 estadoAtual
93             FROM quarto q
94             WHERE q.estadoAtual != 'INATIVO'
95             ORDER BY q.numero
96             """
97
98             self.cursor.execute(query)
99             quartos = self.cursor.fetchall()
100             return quartos
101
102         except mysql.connector.Error as err:
103             print(f"Erro ao listar quartos ativos: {err}")
104             return []
105
106     def criar(self, quarto):
107         """Cria um novo quarto"""
108         try:
109             query = """
110             INSERT INTO quarto (numero, tipo, precoBase,
111 estadoAtual)
112             VALUES (%s, %s, %s, %s)
113             """
114
115             valores = (
116                 quarto['numero'],
117                 quarto['tipo'],
118                 quarto['precoBase'],
119                 quarto.get('estadoAtual', 'DISPONIVEL')
120             )
```

```
117         self.cursor.execute(query, valores)
118         self.connection.commit()
119         return self.cursor.lastrowid
120
121     except mysql.connector.Error as err:
122         print(f"Erro ao criar quarto: {err}")
123         raise
124
125     def eliminar(self, numero):
126         """Elimina um quarto"""
127         try:
128             query = "DELETE FROM quarto WHERE numero = %s"
129             self.cursor.execute(query, (numero,))
130             self.connection.commit()
131             return self.cursor.rowcount > 0
132
133         except mysql.connector.Error as err:
134             print(f"Erro ao eliminar quarto: {err}")
135             raise
136
```

Código 7: Classe Repositório Quarto SQL

```
1 import mysql.connector
2 from datetime import datetime
3
4 class RepositorioReservaSQL:
5
6     def __init__(self):
7         # Configuração da conexão com o MySQL
8         self.connection = mysql.connector.connect(
9             host='localhost',
10            user='root',
11            password='root',
12            database='hotel'
13        )
14        self.cursor = self.connection.cursor(dictionary=True)
15
16    def verificar_disponibilidade(self, numero_quarto,
17        data_entrada, data_saida):
18        """Verifica se o quarto está disponível para as datas
19        solicitadas"""
20        try:
21            query = """
22            SELECT COUNT(*) as count
23            FROM reserva r
24            WHERE r.numeroQuarto = %s
25            AND r.estado NOT IN ('CANCELADA', 'FINALIZADA')
26            AND (
```



```
25         (%s BETWEEN r.dataEntradaPrev AND DATE_SUB(r.  
dataSaidaPrev, INTERVAL 1 DAY))  
26         OR (%s BETWEEN DATE_ADD(r.dataEntradaPrev,  
INTERVAL 1 DAY) AND r.dataSaidaPrev)  
27         OR (r.dataEntradaPrev BETWEEN %s AND DATE_SUB  
(%s, INTERVAL 1 DAY))  
28     )  
29     """  
30  
31     self.cursor.execute(query, (  
32         numero_quarto,  
33         data_entrada,  
34         data_saida,  
35         data_entrada,  
36         data_saida  
37     ))  
38  
39     result = self.cursor.fetchone()  
40     return result['count'] == 0 # Disponível se  
count = 0  
41  
42     except mysql.connector.Error as err:  
43         print(f"Erro ao verificar disponibilidade: {err}")  
44     )  
45     return False  
46  
47     def criar(self, reserva):  
48         """Cria uma nova reserva"""  
49         try:  
50             query = """  
51             INSERT INTO reserva  
52             (dataEntradaPrev, dataSaidaPrev, estado,  
53             valorTotalPrev,  
54             notas, idHospede, numeroQuarto, dataCriacao)  
55             VALUES (%s, %s, %s, %s, %s, %s, %s, %s)  
56             """  
57             valores = (  
58                 reserva['dataEntradaPrev'],  
59                 reserva['dataSaidaPrev'],  
60                 reserva['estado'],  
61                 reserva['valorTotalPrev'],  
62                 reserva['notas'],  
63                 reserva['idHospede'],  
64                 reserva['numeroQuarto'],  
65                 datetime.now().strftime('%Y-%m-%d %H:%M:%S')  
66             )  
67             self.cursor.execute(query, valores)
```

```
68         self.connection.commit()
69
70         return self.cursor.lastrowid
71
72     except mysql.connector.Error as err:
73         print(f"Erro ao criar reserva: {err}")
74         raise
75
76     def obter_lista(self):
77         """Obtém lista de todas as reservas"""
78         try:
79             query = """
80             SELECT r.*, h.nomeCompleto as nomeHospede, q.tipo
81             as tipoQuarto
82             FROM reserva r
83             JOIN hospede h ON r.idHospede = h.idHospede
84             JOIN quarto q ON r.numeroQuarto = q.numero
85             ORDER BY r.dataCriacao DESC
86             """
87
88             self.cursor.execute(query)
89             reservas = self.cursor.fetchall()
90             return reservas
91
92         except mysql.connector.Error as err:
93             print(f"Erro ao obter lista de reservas: {err}")
94             return []
95
96     def obter(self, id_reserva):
97         """Obtém uma reserva específica"""
98         try:
99             query = """
100             SELECT r.*, h.nomeCompleto as nomeHospede, q.tipo
101             as tipoQuarto
102             FROM reserva r
103             JOIN hospede h ON r.idHospede = h.idHospede
104             JOIN quarto q ON r.numeroQuarto = q.numero
105             WHERE r.idReserva = %s
106             """
107
108             self.cursor.execute(query, (id_reserva,))
109             reserva = self.cursor.fetchone()
110             return reserva
111
112         except mysql.connector.Error as err:
113             print(f"Erro ao obter reserva: {err}")
114             return None
115
116     def atualizar_estado(self, id_reserva, novo_estado):
```

```
115     """Atualiza o estado de uma reserva"""
116     try:
117         query = """
118             UPDATE reserva
119             SET estado = %s
120             WHERE idReserva = %s
121             """
122
123         self.cursor.execute(query, (novo_estado,
124 id_reserva))
125         self.connection.commit()
126         return self.cursor.rowcount > 0
127
128     except mysql.connector.Error as err:
129         print(f"Erro ao atualizar estado da reserva: {err
130 }")
131         return False
132
133     def obter_por_hospede(self, id_hospede):
134         """Obtém reservas de um hóspede específico"""
135         try:
136             query = """
137             SELECT r.*, q.tipo as tipoQuarto
138             FROM reserva r
139             JOIN quarto q ON r.numeroQuarto = q.numero
140             WHERE r.idHospede = %s
141             ORDER BY r.dataCriacao DESC
142             """
143
144             self.cursor.execute(query, (id_hospede,))
145             reservas = self.cursor.fetchall()
146             return reservas
147
148         except mysql.connector.Error as err:
149             print(f"Erro ao obter reservas do hóspede: {err}")
150
151     )
152
153     return []
```

Código 8: Classe Repositório Reserva SQL

2.4 Domínio

```
1 import datetime
2 from acesso_dados.repositorio_quarto_sql import
   RepositorioQuartoSQL
3 from acesso_dados.repositorio_reserva_sql import
   RepositorioReservaSQL
```

```
4 from acesso_dados.repositorio_consumo_sql import
    RepositorioConsumoSQL
5
6
7 class ServicoReservas:
8
9     def __init__(self):
10         self.rep_quartos = RepositorioQuartoSQL()
11         self.rep_reservas = RepositorioReservaSQL()
12         self.rep_consumos = RepositorioConsumoSQL()
13         self.precos_servicos = self._carregar_precos_servicos
14         ()
15
16     def _carregar_precos_servicos(self):
17         """Carrega os preços dos serviços do banco de dados"""
18         "
19         try:
20             servicos_db = self.rep_consumos.listar_consumos()
21             precos = {}
22             for servico in servicos_db:
23                 precos[servico['nomeServico']] = float(
24                     servico['preco'])
25             return precos
26         except Exception as e:
27             return "Erro ao carregar preços dos serviços"
28
29     def atualizar_precos_servicos(self):
30         """Atualiza os preços dos serviços do banco de dados"""
31         "
32         self.precos_servicos = self._carregar_precos_servicos
33         ()
34         return self.precos_servicos
35
36     def check_disp_quartos(self, tipo_quarto=None):
37         """Verifica disponibilidade de quartos"""
38         try:
39             if tipo_quarto:
40                 return self.rep_quartos.listar_por_tipo(
41                     tipo_quarto)
42             else:
43                 return self.rep_quartos.listar_disponiveis()
44         except Exception as e:
45             print(f"Erro ao verificar disponibilidade: {e}")
46             return []
47
48     def calc_valor_total_reserva(self, dados_reserva):
49         """Calcula valor total da reserva"""
50         try:
```

```
46         # Obter informações do quarto
47         if dados_reserva.get('numeroQuarto'):
48             quarto = self.rep_quartos.obter_quarto(
dados_reserva['numeroQuarto'])
49         else:
50             # Se não especificar número, assume o
primeiro disponível do tipo
51             quartos_tipo = self.rep_quartos.
listar_por_tipo(dados_reserva['tipoQuarto'])
52             if not quartos_tipo:
53                 raise ValueError(f"Não há quartos disponí
veis do tipo {dados_reserva['tipoQuarto']}")
54             quarto = quartos_tipo[0]
55             dados_reserva['numeroQuarto'] = quarto['
numero']
56
57         if not quarto:
58             raise ValueError(f"Quarto não encontrado: {
dados_reserva.get('numeroQuarto')}")
59
60         # Calcular número de dias
61         data_entrada = datetime.datetime.strptime(
dados_reserva['dataEntradaPrev'], '%Y-%m-%d').date()
62         data_saida = datetime.datetime.strptime(
dados_reserva['dataSaidaPrev'], '%Y-%m-%d').date()
63         numero_dias = (data_saida - data_entrada).days
64
65         if numero_dias <= 0:
66             raise ValueError("Número de dias deve ser
positivo")
67
68         # Calcular valor base
69         valor_base = quarto['precoBase'] * numero_dias
70
71         # Adicionar serviços adicionais
72         valor_servicos = 0
73         if dados_reserva.get('servicosAdicionais'):
74             servico = dados_reserva['servicosAdicionais']
75             if servico in self.precos_servicos:
76                 valor_servicos = self.precos_servicos[
servico] * numero_dias
77             else:
78                 # Tenta encontrar por nome aproximado (
case-insensitive)
79                 servico_lower = servico.lower()
80                 for nome_servico, preco in self.
precos_servicos.items():
81                     if servico_lower in nome_servico.
lower() or nome_servico.lower() in servico_lower:
```

```
82         valor_servicos = preco *
numero_dias
83         break
84
85     # Calcular total
86     valor_total = valor_base + valor_servicos
87
88     return {
89         'valorBase': valor_base,
90         'valorServicos': valor_servicos,
91         'valorTotal': valor_total,
92         'numeroDias': numero_dias,
93         'quarto': quarto
94     }
95
96     except Exception as e:
97         print(f"Erro ao calcular valor total: {e}")
98         raise
99
100     def criar_reserva(self, dados_reserva):
101         """Cria uma nova reserva"""
102         try:
103             # Verificar disponibilidade do quarto
104             if not self.rep_reservas.
verificar_disponibilidade(
105                 dados_reserva['numeroQuarto'],
106                 dados_reserva['dataEntradaPrev'],
107                 dados_reserva['dataSaidaPrev']
108             ):
109                 raise ValueError(f"Quarto {dados_reserva['
numeroQuarto']} não está disponível para as datas
selecionadas")
110
111             # Calcular valor total
112             calculo_valor = self.calc_valor_total_reserva(
dados_reserva)
113
114             # Preparar dados para a reserva
115             reserva_data = {
116                 'dataEntradaPrev': dados_reserva['
dataEntradaPrev'],
117                 'dataSaidaPrev': dados_reserva['dataSaidaPrev
'],
118                 'estado': 'CONFIRMADA',
119                 'valorTotalPrev': calculo_valor['valorTotal'
],
120                 'notas': dados_reserva.get('notas', ''),
121                 'idHospede': dados_reserva['idHospede'],
122                 'numeroQuarto': dados_reserva['numeroQuarto']
```

```
123         }
124
125         # Criar reserva no banco de dados
126         id_reserva = self.rep_reservas.criar(reserva_data
127     )
128
129     if id_reserva:
130         # Atualizar estado do quarto para RESERVADO
131         self.rep_quartos.atualizar_estado(
132             dados_reserva['numeroQuarto'],
133             'RESERVADO'
134         )
135
136         return {
137             'idReserva': id_reserva,
138             'valorTotalPrev': calculo_valor['
139 valorTotal'],
140             'numeroDias': calculo_valor['numeroDias'
141 ],
142             'numeroQuarto': dados_reserva['
143 numeroQuarto'],
144             'tipoQuarto': calculo_valor['quarto']['
145 tipo'],
146             'precosServicos': self.precos_servicos
147         }
148     else:
149         raise ValueError("Erro ao criar reserva no
150 banco de dados")
151
152     except Exception as e:
153         print(f"Erro ao criar reserva: {e}")
154         raise
155
156     def get_lista_hosp_ativos(self):
157         # Obtém lista de hóspedes ativos
158         raise NotImplementedError
159
160     def calcular_total_conta(self):
161         # Calcula total da conta
162         raise NotImplementedError
163
164     def ajustar_datas(self):
165         # Ajusta datas da reserva
166         raise NotImplementedError
167
168     def adicionar_consumos(self):
169         # Adiciona consumos à reserva depois da criação
170         raise NotImplementedError
```

```
166     def atualizar_conta(self):
167         # Atualiza conta da reserva
168         raise NotImplementedError
169
170     def obter_lista_reserva(self):
171         # Obtém lista de reservas
172         raise NotImplementedError
173
174     def get_disponibilidade(self):
175         # Obtém disponibilidade
176         raise NotImplementedError
```

Código 9: Classe Serviço Reserva

```
1 from acesso_dados.repositorio_hospede_sql import
   RepositorioHospedeSQL
2
3 class ServicoHospede:
4     def __init__(self):
5         self.rep_hosp = RepositorioHospedeSQL()
6
7     def consulta_hospede(self, mail):
8         """Consulta hóspede por email"""
9         try:
10             hospede = self.rep_hosp.obter_hospede_por_email(
mail)
11             if hospede:
12                 return {
13                     'success': True,
14                     'hospede': hospede
15                 }
16             else:
17                 return {
18                     'success': False,
19                     'message': 'Hóspede não encontrado'
20                 }, 404
21         except Exception as e:
22             return {
23                 'success': False,
24                 'message': f'Erro ao consultar hóspede: {str(
e)}'
25             }, 500
26
27     def registrar_doc(self, num_doc):
28         """Regista documento do hóspede"""
29         try:
30             existe = self.rep_hosp.verificar_documento(
num_doc)
31             return {
```



```
32         'success': True,
33         'existe': existe
34     }
35     except Exception as e:
36         return {
37             'success': False,
38             'message': f'Erro ao verificar documento: {
str(e)}'
39         }, 500
40
41     def carregar_hospede(self):
42         """Carrega dados do hóspede"""
43         # Implementação depende do contexto
44         pass
45
46     def registrar_hospede(self, dados_hospede):
47         """Regista novo hóspede"""
48         try:
49             # Verificar se documento já existe
50             existe = self.rep_hosp.verificar_documento(
dados_hospede['numDocumento'])
51
52             if existe:
53                 return {
54                     'success': False,
55                     'message': 'Documento já está registrado'
56                 }, 409 # Conflict
57
58             # Registrar hóspede
59             resultado = self.rep_hosp.registrar(dados_hospede)
60
61             return resultado
62
63         except Exception as e:
64             return {
65                 'success': False,
66                 'message': f'Erro ao registrar hóspede: {str(e)
}}'
67         }, 500
```

Código 10: Classe Serviço Hóspede

```
1 class Hospede:
2     def __init__(self):
3         self.nome_completo = ""
4         self.num_documento = ""
5         self.validade_doc = None
6         self.birth_date = None
7         self.nacionalidade = ""
```

```
8         self.email = ""
9         self.telemovel = ""
10        self.morada = ""
11        self.preferencias = ""
12        self.historico = []
13
14    def get_nome_completo(self):
15        return self.nome_completo
16
17    def set_nome_completo(self, nome_completo):
18        self.nome_completo = nome_completo
19
20    def get_num_documento(self):
21        return self.num_documento
22
23    def set_num_documento(self, num_documento):
24        self.num_documento = num_documento
25
26    def get_validade_doc(self):
27        return self.validade_doc
28
29    def set_validade_doc(self, validade_doc):
30        self.validade_doc = validade_doc
31
32    def get_birth_date(self):
33        return self.birth_date
34
35    def set_birth_date(self, birth_date):
36        self.birth_date = birth_date
37
38    def get_nacionalidade(self):
39        return self.nacionalidade
40
41    def set_nacionalidade(self, nacionalidade):
42        self.nacionalidade = nacionalidade
43
44    def get_email(self):
45        return self.email
46
47    def set_email(self, email):
48        self.email = email
49
50    def get_telemovei(self):
51        return self.telemovei
52
53    def set_telemovei(self, telemovel):
54        self.telemovei = telemovel
55
56    def get_morada(self):
```

```
57         return self.morada
58
59     def set_morada(self, morada):
60         self.morada = morada
61
62     def get_preferencias(self):
63         return self.preferencias
64
65     def set_preferencias(self, preferencias):
66         self.preferencias = preferencias
67
68     def get_historico(self):
69         return self.historico
70
71     def set_historico(self, historico):
72         self.historico = historico
```

Código 11: Entidade Hóspede

```
1 class Reserva:
2     """Classe Reserva
3
4     Representa o conceito de Reserva no contexto do projeto
5     """
6
7     def __init__(self):
8         self._id_reserva = None
9         self._data_criacao = None
10        self._data_entrada_prev = None
11        self._data_saida_prev = None
12        self._estado = None
13        self._valor_total_prev = None
14        self._notas = None
15
16        # GETS E SETS
17
18        def get_id_reserva(self):
19            return self._id_reserva
20
21        def set_id_reserva(self, id_reserva):
22            self._id_reserva = id_reserva
23
24        def get_data_criacao(self):
25            return self._data_criacao
26
27        def set_data_criacao(self, data_criacao):
28            self._data_criacao = data_criacao
29
30        def get_data_entrada_prev(self):
```

```
31         return self._data_entrada_prev
32
33     def set_data_entrada_prev(self, data_entrada_prev):
34         self._data_entrada_prev = data_entrada_prev
35
36     def get_data_saida_prev(self):
37         return self._data_saida_prev
38
39     def set_data_saida_prev(self, data_saida_prev):
40         self._data_saida_prev = data_saida_prev
41
42     def get_estado(self):
43         return self._estado
44
45     def set_estado(self, estado):
46         self._estado = estado
47
48     def get_valor_total_prev(self):
49         return self._valor_total_prev
50
51     def set_valor_total_prev(self, valor_total_prev):
52         self._valor_total_prev = valor_total_prev
53
54     def get_notas(self):
55         return self._notas
56
57     def set_notas(self, notas):
58         self._notas = notas
```

Código 12: Entidade Reserva

```
1 from enum import Enum
2
3 class EstadoReserva(Enum):
4     CONFIRMADA = "CONFIRMADA"
5     CHECKIN_EFETUADO = "CHECKIN_EFETUADO"
6     CHECKOUT_EFETUADO = "CHECKOUT_EFETUADO"
7     CANCELADA = "CANCELADA"
```

Código 13: Enumerado Estado da Reserva

```
1 class DadosHospedes:
2
3     def __init__(self):
4         self._nome_completo = None
5         self._num_documento = None
6         self._validade_doc = None
7         self._birth_date = None
8         self._nacionalidade = None
```

```
9         self._email = None
10        self._telemovel = None
11        self._morada = None
12        self._preferencias = None
13        self._historico = [] # Inicializa com lista vazia
14
15        # Getters (sem setters, porque é read_only como indicado
16        # nos diagramas)
17
18        def get_nome_completo(self):
19            return self._nome_completo
20
21        def get_num_documento(self):
22            return self._num_documento
23
24        def get_validade_doc(self):
25            return self._validade_doc
26
27        def get_birth_date(self):
28            return self._birth_date
29
30        def get_nacionalidade(self):
31            return self._nacionalidade
32
33        def get_email(self):
34            return self._email
35
36        def get_telemovel(self):
37            return self._telemovel
38
39        def get_morada(self):
40            return self._morada
41
42        def get_preferencias(self):
43            return self._preferencias
44
45        def get_historico(self):
46            return self._historico
```

Código 14: Dados do Hóspede

```
1 class DadosReserva:
2
3     def __init__(self):
4         self._id_reserva = None
5         self._data_criacao = None
6         self._data_entrada_prev = None
7         self._data_saida_prev = None
8         self._estado = None
```

```
9         self._valor_total_prev = None
10        self._notas = None
11
12        # Getters (sem setters, porque é read_only como indicado
13        # nos diagramas)
14
15        def get_id_reserva(self):
16            return self._id_reserva
17
18        def get_data_criacao(self):
19            return self._data_criacao
20
21        def get_data_entrada_prev(self):
22            return self._data_entrada_prev
23
24        def get_data_saida_prev(self):
25            return self._data_saida_prev
26
27        def get_estado(self):
28            return self._estado
29
30        def get_valor_total_prev(self):
31            return self._valor_total_prev
32
33        def get_notas(self):
34            return self._notas
```

Código 15: Dados da Reserva

```
1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3
4 from acesso_dados.repositorio_hospede_sql import
5     RepositorioHospedeSQL
6 from dominio.api.rotas import Rotas
7 from dominio.servicos.servico_hospede import ServicoHospede
8 from dominio.servicos.servico_reservas import ServicoReservas
9
10 app = Flask(__name__)
11 CORS(app)
12
13 # repositório e serviço
14 rep_hosp = RepositorioHospedeSQL()
15 servico_hospede = ServicoHospede()
16 servico_reservas = ServicoReservas()
17
18 # Instanciar rotas
19 rotas = Rotas(servico_hospede)
```

```
20
21 @app.route('/api/registrar_hosp', methods=['POST'])
22 def registrar_hosp():
23     """Endpoint para registrar hóspede"""
24     try:
25         dados = request.get_json()
26         if not dados:
27             return jsonify({
28                 'success': False,
29                 'message': 'Nenhum dado fornecido'
30             }), 400
31
32         resultado = rotas.registrar_hospede(dados)
33
34         if isinstance(resultado, tuple) and len(resultado) ==
35         2:
36             return jsonify(resultado[0]), resultado[1]
37         else:
38             return jsonify(resultado)
39     except Exception as e:
40         return jsonify({
41             'success': False,
42             'message': f'Erro no servidor: {str(e)}'
43         }), 500
44
45 @app.route('/api/consulta_hospede', methods=['GET'])
46 def consulta_hospede():
47     """Endpoint para consultar hóspede por email"""
48     try:
49         email = request.args.get('email')
50
51         if not email:
52             return jsonify({
53                 'success': False,
54                 'message': 'Parâmetro email é obrigatório'
55             }), 400
56
57         resultado = rotas.consultar_hospede(email)
58
59         if isinstance(resultado, tuple) and len(resultado) ==
60         2:
61             return jsonify(resultado[0]), resultado[1]
62         else:
63             return jsonify(resultado)
64     except Exception as e:
65         return jsonify({
66             'success': False,
```

```
67         'message': f'Erro no servidor: {str(e)}'
68     }, 500
69
70
71 @app.route('/api/confirm_reserva', methods=['POST'])
72 def confirm_reserva():
73     try:
74         dados = request.get_json()
75
76         # Validar dados obrigatórios
77         campos_obrigatorios = [
78             'idHospede', 'dataEntradaPrev', 'dataSaidaPrev'
79         ]
80
81         for campo in campos_obrigatorios:
82             if campo not in dados or not dados[campo]:
83                 return jsonify({
84                     'success': False,
85                     'message': f'Campo obrigatório ausente: {campo}'
86                 }), 400
87
88         # Se não especificar número do quarto, mas especificar
89         # tipo
90         if not dados.get('numeroQuarto') and dados.get('
91         tipoQuarto'):
92             # procurar quarto disponível do tipo especificado
93             quartos_disponiveis = servico_reservas.
94             check_disp_quartos(dados['tipoQuarto'])
95             if not quartos_disponiveis:
96                 return jsonify({
97                     'success': False,
98                     'message': f'Não há quartos disponíveis do tipo {
99                     dados["tipoQuarto"]}'
100                 }), 400
101
102             # usa primeiro quarto disponível
103             dados['numeroQuarto'] = quartos_disponiveis[0]['numero'
104             ]
105         elif dados.get('numeroQuarto'):
106             # Verifica se o quarto específico está disponível
107             quarto = servico_reservas.rep_quartos.obter_quarto(
108             dados['numeroQuarto'])
109             if not quarto:
110                 return jsonify({
111                     'success': False,
112                     'message': f'Quarto {dados["numeroQuarto"]} não
113                     encontrado'
114                 }), 400
115
116             # Se o quarto estiver disponível, reservar
```



```
109         if quarto['estadoAtual'] != 'DISPONIVEL':
110             return jsonify({
111                 'success': False,
112                 'message': f'Quarto {dados["numeroQuarto"]} não est
113                 á disponível'
114             }), 400
115
116         # Criar reserva
117         resultado = servico_reservas.criar_reserva(dados)
118
119         return jsonify({
120             'success': True,
121             'idReserva': resultado['idReserva'],
122             'valorTotalPrev': resultado['valorTotalPrev'],
123             'numeroDias': resultado['numeroDias'],
124             'numeroQuarto': resultado['numeroQuarto'],
125             'tipoQuarto': resultado['tipoQuarto'],
126             'message': 'Reserva confirmada com sucesso'
127         })
128     except ValueError as e:
129         return jsonify({
130             'success': False,
131             'message': str(e)
132         }), 400
133     except Exception as e:
134         return jsonify({
135             'success': False,
136             'message': f'Erro interno: {str(e)}'
137         }), 500
138
139 @app.route('/api/quartos_disponiveis', methods=['GET'])
140 def quartos_disponiveis():
141     try:
142         tipo = request.args.get('tipo')
143         if tipo:
144             quartos = servico_reservas.check_disp_quartos(tipo)
145         else:
146             quartos = servico_reservas.check_disp_quartos()
147
148         return jsonify(quartos)
149
150     except Exception as e:
151         return jsonify({
152             'success': False,
153             'message': f'Erro ao obter quartos disponíveis: {str(e)}
154         }'), 500
155
```

```
156 if __name__ == '__main__':  
157     app.run(debug=True, port=5008)
```

Código 16: Adaptador Servidor (API)

```
1 class Rotas:  
2     def __init__(self, servico_hospede):  
3         self.servico_hospede = servico_hospede  
4  
5     def registrar_hospede(self, dados_hospede):  
6         """Rota para registrar um novo hóspede"""  
7         try:  
8             # Validar dados obrigatórios  
9             campos_obrigatorios = ['nomeCompleto', '  
10                                     'birthDate', 'nacionalidade'  
11                                     ', 'email', 'telemovel']  
12  
13             for campo in campos_obrigatorios:  
14                 if campo not in dados_hospede or not  
15                 dados_hospede[campo]:  
16                     return {  
17                         'success': False,  
18                         'message': f'Campo obrigatório  
19                         faltando: {campo}',  
20                         }, 400  
21  
22             # serviço para registrar hóspede  
23             resultado = self.servico_hospede.registrar_hospede  
24             (dados_hospede)  
25  
26             return resultado  
27  
28         except Exception as e:  
29             return {  
30                 'success': False,  
31                 'message': f'Erro interno do servidor: {str(e)  
32                 }',  
33                 }, 500  
34  
35     def consultar_hospede(self, email):  
36         """Rota para consultar hóspede por email"""  
37         try:  
38             if not email:  
39                 return {  
40                     'success': False,  
41                     'message': 'Email é obrigatório'  
42                 }, 400
```

```
39         # serviço para consultar hóspede
40         resultado = self.servico_hospede.consulta_hospede
41         (email)
42         return resultado
43
44     except Exception as e:
45         return {
46             'success': False,
47             'message': f'Erro interno do servidor: {str(e)}'
48         }, 500
```

Código 17: Rotas da API (API)

Capítulo 3

Testes

```
1 from dominio.servicos.servico_reservas import ServicoReservas
2 from dominio.servicos.servico_hospede import ServicoHospede
3 import time
4
5 def teste_criar_reserva():
6     """Função para teste de criação de reserva"""
7     print("=== TESTE DE CRIAÇÃO DE RESERVA ===")
8
9     # Inicializar serviços
10    servico_hospede = ServicoHospede()
11    servico_reservas = ServicoReservas()
12
13    # 1. Primeiro, criar um hóspede de teste
14    print("\n--- Criando hóspede de teste ---")
15    timestamp = int(time.time())
16    num_doc = f"TESTERES{timestamp}"
17    email = f"teste.reserva{timestamp}@email.com"
18
19    dados_hospede = {
20        'nomeCompleto': 'Teste Reserva Automática',
21        'numDocumento': num_doc,
22        'validadeDoc': '2025-12-31',
23        'birthDate': '1990-05-15',
24        'nacionalidade': 'pt',
25        'email': email,
26        'telemovel': '912345678',
27        'preferencias': 'Teste automático de reserva'
28    }
29
30    print(f"Registando hóspede com documento: {num_doc}")
31    print(f"Email: {email}")
32
```

```
33     resultado_hospede = servico_hospede.registar_hospede(
34         dados_hospede)
35
36     if not resultado_hospede.get('success'):
37         print(f"FALHA ao criar hóspede: {resultado_hospede.
38             get('message')}")
39         return resultado_hospede
40
41     id_hospede = resultado_hospede.get('idHospede')
42     print(f"SUCESSO! ID do hóspede: {id_hospede}")
43
44     # 2. Verificar quartos disponíveis
45     print("\n--- Verificando quartos disponíveis ---")
46     quartos_disponiveis = servico_reservas.check_disp_quartos
47     ()
48
49     if not quartos_disponiveis:
50         print("Nenhum quarto disponível para teste!")
51         return {
52             'success': False,
53             'message': 'Nenhum quarto disponível'
54         }
55
56     # Usar o primeiro quarto disponível
57     quarto_selecionado = quartos_disponiveis[0]
58     print(f"Quarto selecionado: {quarto_selecionado['numero
59         ']} - {quarto_selecionado['tipo']}")
60     print(f"Preço base: {quarto_selecionado['precoBase']} /
61         noite")
62
63     # 3. Preparar dados da reserva
64     print("\n--- Preparando dados da reserva ---")
65
66     # Datas de teste (amanhã para 3 dias depois)
67     from datetime import datetime, timedelta
68     hoje = datetime.now()
69     entrada = (hoje + timedelta(days=1)).strftime('%Y-%m-%d')
70     saida = (hoje + timedelta(days=4)).strftime('%Y-%m-%d')
71
72     dados_reserva = {
73         'idHospede': id_hospede,
74         'numeroQuarto': quarto_selecionado['numero'],
75         'tipoQuarto': quarto_selecionado['tipo'],
76         'dataEntradaPrev': entrada,
77         'dataSaidaPrev': saida,
78         'servicosAdicionais': 'Café da Manhã', # Serviço
79         opcional
80         'notas': 'Reserva criada por teste automático'
81     }
```

```
76
77     print(f"Data de entrada: {entrada}")
78     print(f"Data de saída: {saida}")
79     print(f"Duração: 3 noites")
80     print(f"Serviço adicional: {dados_reserva['servicosAdicionais']}")
81
82     # 4. Calcular valor total (opcional, para verificação)
83     print("\n--- Calculando valor total ---")
84     try:
85         calculo = servico_reservas.calc_valor_total_reserva(
86             dados_reserva)
87         print(f"Valor base: {calculo['valorBase']} ")
88         print(f"Valor serviços: {calculo['valorServicos']}")
89         print(f"VALOR TOTAL: {calculo['valorTotal']} ")
90         print(f"Número de dias: {calculo['numeroDias']}")
91     except Exception as e:
92         print(f"Erro no cálculo: {e}")
93
94     # 5. Criar a reserva
95     print("\n--- Criando reserva ---")
96
97     try:
98         resultado_reserva = servico_reservas.criar_reserva(
99             dados_reserva)
100
101         print(f"\nSUCESSO!")
102         print(f"ID da reserva: {resultado_reserva.get('idReserva')}")
103         print(f"Valor total: {resultado_reserva.get('valorTotalPrev')} ")
104         print(f"Número de dias: {resultado_reserva.get('numeroDias')}")
105         print(f"Quarto: {resultado_reserva.get('numeroQuarto')}")
106         print(f"Tipo de quarto: {resultado_reserva.get('tipoQuarto')}")
107         print(f"Mensagem: Reserva criada com sucesso!")
108
109     # 6. Verificar se o quarto foi atualizado para
110     RESERVADO
111     print("\n--- Verificando estado do quarto ---")
112     quarto_atual = servico_reservas.rep_quartos.obter_quarto(quarto_selecionado['numero'])
113     print(f"Estado atual do quarto: {quarto_atual.get('estadoAtual')}")
114
115     return {
```

```

113         'success': True,
114         'idHospede': id_hospede,
115         'reserva': resultado_reserva,
116         'message': 'Reserva criada com sucesso'
117     }
118
119     except ValueError as e:
120         print(f"\nFALHA - Erro de validação!")
121         print(f"    Mensagem: {str(e)}")
122         return {
123             'success': False,
124             'message': str(e),
125             'tipo': 'validacao'
126         }
127     except Exception as e:
128         print(f"\nFALHA - Erro geral!")
129         print(f"    Mensagem: {str(e)}")
130         import traceback
131         traceback.print_exc()
132         return {
133             'success': False,
134             'message': str(e),
135             'tipo': 'geral'
136         }
137
138
139 if __name__ == '__main__':
140     resultado = teste_criar_reserva()
141
142     print("\n" + "="*50)
143     print("RESUMO DO TESTE:")
144     print(f"Status: {'SUCESSO' if resultado.get('success')
145     else 'FALHA'}")
146     if resultado.get('success'):
147         print(f"ID Hóspede: {resultado.get('idHospede')}")
148         print(f"ID Reserva: {resultado.get('reserva', {}).get(
149         ('idReserva'))}")
150     else:
151         print(f"Mensagem de erro: {resultado.get('message')}")
152 )

```

Código 18: Módulo de teste para criar reserva

```

1 from dominio.servicos.servico_hospede import ServicoHospede
2
3
4 def teste_registo():
5     """Função para teste"""
6     print("=== TESTE DE REGISTO DE HÓSPEDE ===")

```

```
7
8     servico = ServicoHospede()
9
10    # Documento único baseado no timestamp para evitar
    conflitos nos testes
11    import time
12    timestamp = int(time.time())
13    num_doc = f"TESTE{timestamp}"
14    email = f"teste{timestamp}@email.com"
15
16    dados_hospede = {
17        'nomeCompleto': 'Teste Automático',
18        'numDocumento': num_doc,
19        'validadeDoc': '2025-12-31',
20        'birthDate': '1995-01-01',
21        'nacionalidade': 'pt',
22        'email': email,
23        'telemovel': '911111111',
24        'preferencias': 'Teste automático do sistema'
25    }
26
27    print(f"\nTentando registrar hóspede com documento: {
    num_doc}")
28    print(f"Email: {email}")
29
30    resultado = servico.registrar_hospede(dados_hospede)
31
32    if resultado.get('success'):
33        print(f"\nSUCESSO!")
34        print(f"    Mensagem: {resultado.get('message')}")
35        print(f"    ID do hóspede: {resultado.get('idHospede')}
    ")
36    else:
37        print(f"\nFALHA!")
38        print(f"    Mensagem: {resultado.get('message')}")
39
40    return resultado
41
42 if __name__ == '__main__':
43     teste_registro()
```

Código 19: Módulo de teste para registrar hóspede

```
1 from acesso_dados.repositorio_consumo_sql import
    RepositorioConsumoSQL
2
3 class TesteListarConsumos:
4     def __init__(self):
5         self.repositorio = RepositorioConsumoSQL()
```



```

6
7     def testar_listar_servicos_disponiveis(self):
8         """
9         Testa a função de listar consumos/serviços
10        """
11        print(f"\n{'='*50}")
12        print(f"Teste: Listar Serviços")
13        print(f"{'='*50}")
14
15        try:
16            # Chama a função do repositório
17            servicos = self.repositorio.listar_consumos()
18
19            print(servicos)
20
21            if servicos:
22                print(f"\nEncontrados {len(servicos)} disponí
veeis:")
23                print(f"{'-'*50}")
24
25                for i, servico in enumerate(servicos, 1):
26                    print(f"Serviço #{i}:")
27                    print(f"    ID: {servico['idServico']}")
28                    print(f"    Nome: {servico['nomeServico']}")
29                print(f"    Preço:      {servico['preco']:.2f
}")
30
31            else:
32                print(f"Nenhum quarto disponível encontrado.")
33
34            return servicos
35
36        except Exception as e:
37            print(f"\nErro ao listar quartos disponíveis: {e}
")
38            import traceback
39            traceback.print_exc()
40            return None
41
42    if __name__ == "__main__":
43        tester = TesteListarConsumos()
44        tester.testar_listar_servicos_disponiveis()

```

Código 20: Módulo de teste para listar serviços/consumos

```

1 from acesso_dados.repositorio_quarto_sql import
    RepositorioQuartoSQL

```

```
2
3 class TesteListarQuartosDisp:
4     def __init__(self):
5         self.repositorio = RepositorioQuartoSQL()
6
7     def testar_listar_quartos_disponiveis(self):
8         """
9         Testa a função de listar quartos disponíveis
10        """
11        print(f"\n{'='*50}")
12        print(f"Teste: Listar Quartos Disponíveis")
13        print(f"{'='*50}")
14
15        try:
16            # Chama a função do repositório
17            quartos_disponiveis = self.repositorio.
18            listar_disponiveis()
19
20            #print(quartos_disponiveis)
21
22            if quartos_disponiveis:
23                print(f"\nEncontrados {len(
24                    quartos_disponiveis)} quartos disponíveis:")
25                print(f"{'-'*50}")
26
27                for i, quarto in enumerate(
28                    quartos_disponiveis, 1):
29                    print(f"\nQuarto #{i}:")
30                    print(f"    Número: {quarto['numero']}")
31                    print(f"    Tipo: {quarto['tipo']}")
32                    print(f"    Preço por noite: {quarto['
33                        precoBase']:.2f}")
34
35            else:
36                print(f"Nenhum quarto disponível encontrado.")
37        )
38
39        return quartos_disponiveis
40
41    except Exception as e:
42        print(f"\nErro ao listar quartos disponíveis: {e}
43        ")
44
45        import traceback
46        traceback.print_exc()
47        return None
48
49 if __name__ == "__main__":
50     tester = TesteListarQuartosDisp()
51     tester.testar_listar_quartos_disponiveis()
```

Código 21: Módulo de teste para listar quartos disponíveis

Bibliografia

[Morgado, 2025] Morgado, L. (2025). Documentação de apoio à unidade curricular engenharia de software. Disponibilizado via Moodle. Acedido: 2025.